

ECLIPSE: A Digital Engineering Ecosystem for Collaborative Lunar Base Architecting

Arthur Brunier^{*}, Angelos Lachanas[†], Achal Kikkeri[‡], Zhaoxin Jenny Ma[§], and Mustafa Siddiqui[¶], Ivette Malave^{||}
Georgia Institute of Technology, Atlanta, Georgia, 30332

Designing and architecting lunar base infrastructure requires coordinating multiple independent organizations around a shared mission definition, evaluating complex system interactions before any hardware is deployed, and producing results that are interpretable beyond quantitative metrics alone. This paper presents ECLIPSE, a Digital Engineering Ecosystem developed to address these challenges through a unified, requirements-driven pipeline connecting eight functional blocks: an Authoritative Source of Truth based on MBSE with SysML v2, Module Selection, GIS-based Urban Planning, a Discrete Event Simulation Behavior Engine, a Requirements Verification Loop, CAD Submission, Visualization in NVIDIA Omniverse, and a Collaboration infrastructure built on GitHub and open standards. The pipeline is modular at two levels: the SysML model defines the set of available systems, and Module Selection allows any subset to be activated for a given scenario without modifying the underlying code. All configuration, terrain, simulation, and geometry data flows through a universal JSON backbone, ensuring metadata integrity across organizational boundaries. The pipeline is demonstrated on a Lunar Spaceport-1 ISRU scenario, in which GIS analysis of nine Artemis Candidate Landing Regions identified Malapert Massif as the optimal deployment site, discrete event simulation produced 111.23 kg of liquid oxygen over 27.5 operational hours while maintaining full battery charge, and the resulting mission scenario was played back in NVIDIA Omniverse with each system placed at its terrain-aware location. ECLIPSE demonstrates the feasibility of a collaborative, end-to-end DEE pipeline for lunar base architecting, developed as part of the LUNaverse Grand Collaboration Phase 0.

I. Introduction

A. Background

The last decade has marked a fundamental turning point in the ambition and scale of global lunar exploration programs. Rather than short-duration surface visits, the goal has shifted toward long-term permanence. NASA's Artemis program, supported by the National Space Policy of the United States, explicitly commits to establishing "a permanent human presence on the Moon and beyond" [1], while the National Cislunar Science and Technology Action Plan reinforces this direction by calling on the United States to "advance a robust Cislunar ecosystem" [2]. China has similarly announced plans to land astronauts on the Moon by 2030 [3], signaling that this transition from exploration to permanence is not a single-nation ambition but a defining feature of the next era of space development.

This shift in objectives demands a corresponding shift in engineering complexity. Sustaining a human presence on the lunar surface requires an integrated System of Systems (SoS) in which habitats, power generation networks, In-Situ Resource Utilization (ISRU) facilities, communications infrastructure, and transportation assets must operate continuously and interdependently [4, 5]. A lunar spaceport, serving as the logistical and operational hub for this infrastructure, illustrates precisely the scale of this challenge: its design must simultaneously account for landing safety, terrain stability, illumination conditions, dust dynamics, and proximity to exploitable resources, all of which must be coordinated across multiple organizations and teams [6]. The engineering methods that supported Apollo-era missions, built around isolated simulations and static documentation, are fundamentally insufficient for a challenge of this nature.

^{*}Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

[†]Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

[‡]Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

[§]Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

[¶]Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

^{||}Insert Job Title, Department Name, Address/Mail Stop, and AIAA Member Grade (if any).

Long development cycles, incompatible data formats, and late system integration drive up costs and increase mission risk in ways that become unacceptable when the infrastructure in question must operate reliably on the surface of the Moon.

B. Motivation

Designing and operating a lunar spaceport involves a level of organizational complexity that goes well beyond traditional space mission planning. Each system, whether it is a habitat, an ISRU plant, a power grid, or a communications array, is developed by a different organization, often using different tools, different data formats, and different internal standards. A wide variety of data, each with its own units, formats, and standards, must flow between tools and between organizations throughout the design process. Even within a single organization, multiple versions of the same system model can coexist simultaneously, each reflecting a different design iteration or assumption set. When all of these systems and all of these organizations must be brought together into a coherent mission architecture, the number of possible configurations becomes enormous. Lunar base architecting fundamentally requires stakeholders to work together and integrate their systems, yet the tools and processes currently available were not built with this kind of collaboration in mind. Beyond the technical challenge, collaboration in this domain also raises concerns that cannot be overlooked: intellectual property protection and the handling of sensitive or proprietary data must be safeguarded even within a shared engineering environment.

What is needed is a framework that is modular and scalable by design. A lunar mission is not a fixed configuration: the systems involved, their number, their interdependencies, and the terrain constraints they must satisfy all vary depending on mission objectives. The ideal framework would allow any mission configuration to be assembled directly from its requirements, simulated, and evaluated without rebuilding the underlying process from scratch each time.

Evaluation itself presents a distinct challenge. Understanding how a mission will unfold requires more than a set of performance numbers. Knowing which systems are active at which moment, how resources flow between them, and where operational bottlenecks emerge demands a spatial and temporal picture that quantitative metrics alone cannot provide. Mission planners need to see the mission in context in order to make confident architectural decisions and meaningfully compare alternative configurations

In other engineering domains, Digital Engineering Ecosystems and digital twin frameworks have shown that precisely these kinds of challenges can be addressed. In automotive, defense, and aerospace, integrated model-based environments have enabled large, distributed teams to collaborate on complex systems, evaluate designs through immersive visualization before any hardware is built, and iterate rapidly without losing traceability. Whether these capabilities can be brought to bear on the specific demands of lunar base architecting is the central question motivating this work.

II. Problem Formulation

A. Literature Review

The State of Lunar Mission Architecting

The challenge of designing sustainable lunar infrastructure has attracted significant research effort over the past several years, with contributions spanning system architecture, subsystem design, and mission planning methodologies. The Georgia Tech Cislunar Architecture Study established a System of Systems perspective for sustainable lunar infrastructure development, demonstrating that long-term success on the lunar surface depends on shared information ecosystems and interoperable decision environments [4]. Building on this foundation, the Cislunar Environment Architecting Tabletop Study showed that collaborative modeling and simulation environments allow stakeholders to evaluate trade-offs between scientific, commercial, and operational mission objectives [6].

At the systems level, Boyer et al. developed an integrated parametric framework for designing ISRU capabilities within lunar base architectures, providing a structured approach to sizing and trade space exploration for resource utilization systems [7]. Ramkumar et al. extended this work to the power domain, presenting an integrated MDAO-enabled MBSE approach for sizing lunar power systems that connects architectural decisions to quantitative performance metrics [8]. Mehta et al. further contributed a parametric modeling approach for lunar base architectures with a particular focus on power system alternatives, enabling rapid comparison of design options at the architecture level [9]. At the mission operations level, the Moon Operations for Resupply Payload Handling (MORPH) Grand Challenge

demonstrated how logistics and resupply workflows can be modeled and analyzed for lunar surface scenarios [10]. The Design Environment for Lunar Operations and Systems Architecting (DELOS) Grand Challenge produced a dedicated design environment for lunar surface operations, providing tools for scenario construction and system integration [11]. Most recently, the Permanent Infrastructure and Operations for Next-generation Earth-to-Moon Establishment and Research (PIONEER) Grand Challenge addressed the longer-term challenge of permanent lunar infrastructure, extending the scope of prior work toward sustained operations [12].

The State of Digital Engineering

The U.S. Department of Defense defines Digital Engineering as "an integrated digital approach that uses authoritative digital models as a source of truth to support activities across the product lifecycle" [13]. Building on this foundation, a Digital Engineering Ecosystem (DEE) is defined as the interconnected infrastructure, environment, and methodology, encompassing processes, methods, and tools, used to store, access, analyze, and visualize evolving systems data and models to address the needs of stakeholders [13]. A DEE operates across three layers: the Digital Engineering Infrastructure, which provides the technical backbone of computing, storage, networking, and integration services; the Digital Data, Models, and Analyses layer, which contains the digital artifacts including the authoritative source of truth; and the Digital Engineering Environment, which constitutes the collaborative digital workspace where stakeholders perform engineering activities. Model-Based Systems Engineering (MBSE) provides the formal backbone for this approach, enabling requirements, architecture, behavior, and interfaces to be captured in a single coherent representation that evolves with the system throughout its lifecycle [14].

Digital Engineering Ecosystems extend this model-centric philosophy across entire organizations and supply chains. Digital twins, defined as high-fidelity virtual representations of physical systems that mirror their structure, behavior, and environment, have demonstrated their value in reducing development risk and enabling full-scale experimentation before hardware is built or deployed [15]. In the automotive industry, digital twin environments allow engineers from different companies to test component interactions before physical integration. In aerospace and defense, integrated simulation platforms enable large-scale scenario testing and mission rehearsal, cutting integration costs and enabling forms of collaboration that were previously impractical [13]. The LUNArverse Grand Collaboration Phase 0, led by Georgia Tech, Purdue, The Aerospace Corporation, NVIDIA, and the Open Solar System Foundation, represents the most advanced effort to bring digital engineering, spatial data integration, and common operational simulation capabilities to the lunar domain, establishing an authoritative source of spatial truth and a Common Operational Simulation environment built on open standards [16]. ECLIPSE, the system presented in this paper, is developed as part of this initiative, focusing specifically on digital engineering support for lunar surface infrastructure planning, simulation, and visualization.

B. Research Gap

The works reviewed above point to a clear and specific gap. Lunar mission design has produced increasingly capable methods for architecting surface infrastructure, yet these methods are scenario-specific, time-consuming to configure, and do not support structured multi-stakeholder collaboration with data integrity and traceability guarantees. Digital Engineering Ecosystems, on the other hand, have demonstrated in other domains that exactly these challenges can be addressed through model-centric, collaborative, and visualization-driven environments. Rather than opposing these two bodies of work, the gap lies in their combination: the lunar mission design community has the domain knowledge, and the digital engineering community has the tools, but no existing work has brought them together into a unified, end-to-end pipeline for lunar base architecting.

More specifically, no existing framework combines requirements-driven modularity, multi-stakeholder collaboration through open standards, and visualization-based mission evaluation into a single traceable pipeline applicable across mission types. This is not to say that collaboration or visualization are entirely absent from the field. The Cislunar Environment Architecting Tabletop Study demonstrated that stakeholders can be brought together in a structured exercise, and individual DE tools have been adopted in several lunar design efforts. What is missing is a pipeline in which independent organizations can contribute their systems and data in a structured, non-interfering, and traceable way, starting from mission requirements, and produce a scenario that can be evaluated not only through quantitative metrics but through spatial and temporal visualization. This is the gap that ECLIPSE addresses.

C. Proposed Work: Objectives and Contributions

The objective of this work is to develop ECLIPSE, a collaborative platform for lunar mission generation, simulation, and evaluation that operates from mission requirements through to visualization, and that is designed to be generalizable across mission types without requiring the underlying process to be rebuilt for each new scenario.

Three research questions guide this work. First, how can a lunar scenario generation pipeline be designed to be entirely requirements-driven and modular by design, so that any system configuration can be generated, simulated, and evaluated through a single generalized optimization process that adapts to the mission at hand, rather than requiring a new dedicated process to be built for each scenario? Second, how can independent stakeholders contribute their systems and data into a shared pipeline while guaranteeing traceability, metadata integrity, and non-interference between contributions? Third, how can the evaluation of a lunar mission go beyond quantitative Measures of Effectiveness to provide a spatial and temporal understanding of mission behavior that is directly interpretable by all stakeholders?

ECLIPSE addresses these questions through four primary contributions. It provides an end-to-end requirements-driven pipeline connecting SysML-based system modeling, GIS-based site selection, discrete event simulation, and 3D visualization within a single traceable digital thread. It implements modularity at two levels: at the requirements level, where the pipeline adapts automatically to any mission configuration defined in the system model, and at the execution level, where each component can be activated or deactivated independently without modifying the underlying code. It enables multi-stakeholder collaboration through shared open standards and version control, allowing independent contributors to work in parallel without interfering with one another while maintaining full traceability and intellectual property safeguards. Finally, it replaces purely quantitative mission evaluation with visualization-based assessment in NVIDIA Omniverse, giving all stakeholders a direct spatial and temporal understanding of how the mission unfolds.

III. Architecture

The challenges identified in the Problem Formulation translate into a set of formal requirements for the DEE, spanning six key capability areas: interoperability, scalability, traceability, multi-user support, versioning, and modularity. These requirements drive the derivation of the building blocks of ECLIPSE.

A. Derivation of the Main Building Blocks

Authoritative Source of Truth

The traceability and versioning requirements call for a single, version-controlled, and formally structured representation of the mission architecture that all stakeholders can reference throughout the pipeline: this is the role of the Authoritative Source of Truth block. It defines system structure, requirements, behavior, and interfaces in one consistent model, enabling all downstream activities to reference the same validated information. In a multi-stakeholder workflow, document-centric approaches are difficult to keep synchronized, since requirements, interfaces, and design decisions can diverge across separate files and tools. For this reason, the pipeline adopts MBSE with SysML v2 as the formal modeling backbone. SysML v2 can represent system-of-systems structure, behavior, interfaces, and requirements in a unified way, while linking requirements to model elements through satisfy relationships, enabling traceability from mission requirements to verification and validation. The block takes mission requirements entered directly by the user into the MBSE software as input, and outputs a SysML model that serves as the foundation for all downstream blocks in the pipeline.

Module Selection

The modularity and interoperability requirements call for a pipeline in which the active mission systems can be configured for any given scenario, and in which this configuration propagates automatically to all downstream processes: this is the role of the Module Selection block. This modularity operates at two levels: at the requirements level, the SysML model defines which systems exist in the pipeline, and at the execution level, each system can be activated or deactivated independently. Module Selection enables this second level by allowing the user to define which systems participate in a given scenario, while preserving the interfaces, attributes, and relationships inherited from the SysML model. This makes it straightforward to scale a scenario up or down, from a minimal early-deployment configuration to a fully integrated spaceport, without any structural changes to the pipeline. The block takes the SysML model as input and outputs an enriched data file containing the active systems, their interfaces, and their attributes, which serves as the

data backbone for all downstream blocks in the pipeline. This data exchange is implemented using JSON, adopted as the universal format because it is lightweight, human-readable, natively supported in Python, and capable of preserving the full hierarchy and attributes of the SysML model without information loss.

Urban Planning

Grounding the abstract system configuration in a physically realistic lunar terrain context before simulation is a prerequisite for meaningful mission evaluation. Early spatial decisions shape all downstream outcomes, including infrastructure feasibility, operational connectivity, landing safety, energy access, and surface mobility. Planning upstream reduces uncertainty, improves realism, and enables more informed mission-level decision-making. This is the role of the Urban Planning block within the ECLIPSE pipeline, where GIS-based spatial analysis transforms mission requirements into decision-ready site selection and infrastructure placement recommendations.

The Urban Planning block operates in two sequential phases: site selection and module placement.

In the first phase, site selection, terrain conditions across the nine NASA Artemis Candidate Landing Regions (CLRs) at the lunar south pole are evaluated using a GIS workflow implemented in ArcGIS Pro. Lunar Orbiter Laser Altimeter (LOLA) Digital Elevation Model (DEM) data is used to derive terrain characteristics such as slope, elevation, and terrain stability, while illumination conditions and proximity to Permanently Shadowed Regions (PSRs) are incorporated as additional planning criteria due to their importance for power generation and resource accessibility. These environmental layers are evaluated through a weighted suitability framework that allows candidate sites to be compared across multiple operational constraints simultaneously. The workflow generates terrain suitability heatmaps and ranked site scores, enabling users to identify locations most appropriate for sustained lunar infrastructure deployment. The selected site then becomes the spatial foundation for downstream infrastructure planning, simulation, and visualization activities.

In the second phase, module placement, the active systems selected during the Module Selection step are positioned within the selected site using local terrain constraints and module-specific operational requirements. Slope and illumination heatmaps provide visual guidance for infrastructure placement, helping ensure that systems are located in operationally feasible regions. Different infrastructure elements prioritize different terrain conditions depending on their operational role. For example, landing zones prioritize low-slope terrain, solar infrastructure prioritizes high-illumination areas, and ISRU systems prioritize accessibility to PSRs and traversable terrain. An automated routing algorithm then generates traversable connections between modules while minimizing both travel distance and terrain difficulty, supporting efficient rover mobility and operational connectivity between systems. The resulting infrastructure layout can also be adjusted interactively by the user, enabling rapid evaluation of alternative configurations and mission tradeoffs within the same planning environment.

The Urban Planning block takes the enriched JSON generated during Module Selection as input and outputs an updated JSON containing spatial metadata, including coordinates, orientations, terrain suitability information, and routing paths for each active module. This updated representation is then propagated downstream into the SE Behavior Engine and visualization environment, enabling terrain-aware mission simulation, operational analysis, and spatially grounded mission evaluation within the broader Digital Engineering Ecosystem.

Behavior Engine

To evaluate whether the mission architecture can sustain its operations over time, the pipeline requires a block capable of simulating time-dependent interactions and resource flows between active systems before any hardware is deployed: this is the role of the Behavior Engine. A lunar spaceport contains asynchronous and event-driven operations, including rover arrivals, resource transfers, production cycles, storage thresholds, and power demand variations. These interactions are difficult to capture with static models alone, while continuous simulation is not always the most natural representation for operations that occur as discrete events. For this reason, the architecture adopts Discrete Event Simulation (DES) implemented with SimPy in Python. DES is selected because it directly represents event-based processes, resource contention, queues, and system state changes over time. SimPy is lightweight, open-source, and directly compatible with the JSON-based pipeline. The block takes the enriched JSON from Urban Planning as input and outputs a results JSON containing the final state of all systems, together with a log JSON containing the complete mission time history, both of which feed into the downstream blocks.

Requirements Verification Loop

Connecting simulation results back to the requirements defined at the architecture level, and doing so automatically and traceably, is what the Requirements Verification Loop block addresses. Because the Behavior Engine produces a

time-evolving mission history, manual verification would be inefficient and prone to errors, especially when requirements depend on resource levels, system states, timing constraints, or operational thresholds. The verification loop compares the requirements encoded in the SysML model with the log JSON generated by the Behavior Engine. For each requirement, the pipeline checks whether the relevant condition is satisfied at each timestep and records a pass or fail result. This produces a structured verification output that identifies which requirements are satisfied, when violations occur, and which systems are involved. The results feed back into the Authoritative Source of Truth by flagging requirement violations and supporting model updates, closing the digital engineering loop by connecting requirements, simulation behavior, and verification evidence within the same traceable workflow.

CAD Submission

Once the mission systems have been defined, configured, and spatially grounded, their physical geometry must be integrated into the pipeline so that visualization reflects the actual appearance of each active system: this is the role of the CAD Submission block. While Module Selection defines which systems are active and how they are described semantically, CAD Submission connects those active systems to their physical representations. This is important because geometric models may be developed independently by different organizations and must still be linked to the correct system element, metadata, and scenario configuration. The accepted 3D format is determined by the visualization environment, as discussed in the following block, ensuring compatibility with the final inspection platform. The block takes the enriched JSON from Module Selection and the 3D models provided by hardware stakeholders as input, and outputs a final enriched JSON that includes references to the 3D assets of each active system. At this stage, the pipeline has everything it needs to bring the mission to life: a complete system description, a spatial layout, simulation results, and geometry for every active component.

Visualization

With the mission fully defined, simulated, and geometrically represented, the final block transforms all of these outputs into a directly interpretable evaluation of the mission: this is the role of the Visualization block. Numerical outputs can compare alternatives, but they do not always reveal when systems are active, how resources move, where spatial conflicts occur, or where operational bottlenecks emerge. A 3D environment allows stakeholders to inspect the evolving architecture, compare alternatives more intuitively, and connect simulation results with the physical layout of the mission.

A multi-criteria decision-making evaluation was conducted to select the visualization platform, as shown in Table 1. NVIDIA Omniverse was selected with the highest score of 4.55 because of its strong interoperability, scale handling, extensibility, and automation readiness. With the visualization platform defined, the 3D data interoperability standard could be selected. A second evaluation was conducted for this purpose, as shown in Table 2. OpenUSD achieved the highest score of 4.80, owing to its interoperability, extensibility, metadata richness, ability to represent complex hierarchical scenes without information loss, and its native integration within Omniverse. This choice retroactively defines the format required for CAD Submission. The block takes the final enriched JSON and the simulation log as inputs and outputs a 3D mission scenario with an accompanying telemetry dashboard.

Table 1 MCDM evaluation of candidate visualization platforms.

Criterion	Weight	Omniverse	Siemens NX	3DEXPERIENCE	ANSYS Minerva
Interoperability & Integration	30%	5	4	5	4
Performance & Scale Handling	25%	4	5	4	4
Extensibility / Flexibility	20%	5	5	4	4
AI & Automation Readiness	15%	5	4	4	4
Accessibility / Licensing	10%	3	3	3	3
Total	–	4.55	4.35	4.20	3.90

Table 2 MCDM evaluation of candidate 3D data interoperability standards.

Criterion	Weight	OpenUSD	glTF	COLLADA
Interoperability	30%	5	5	2
Performance / Lightweight Handling	10%	5	5	1
Extensibility & Scalability	20%	5	4	1
Fidelity / Data Integrity	20%	4	3	3
Automation & AI Readiness	15%	5	5	2
Adoption / Accessibility	5%	5	5	2
Total	–	4.80	4.40	1.90

Collaboration

Enabling independent organizations to contribute to the same pipeline without interfering with one another, while maintaining full traceability and intellectual property safeguards, is the role of the Collaboration block. Lunar base architecting involves distributed contributors working on different parts of the same digital thread, including mission planners, hardware teams, GIS specialists, simulation developers, and mission evaluators. Without a structured collaboration backbone, parallel contributions could overwrite one another, metadata could be lost, and design decisions would lack an audit trail. GitHub is used as the version-control backbone because it supports isolated development, pull requests as a conflict-resolution mechanism, complete commit history, and GitHub Actions for automated pipeline execution triggered at each commit. Shared open standards, including SysML, JSON, and OpenUSD, further protect metadata integrity across organizational boundaries without requiring stakeholders to use the same proprietary tools. Unlike the other blocks, Collaboration spans the entire pipeline rather than occupying a single position within it, taking stakeholder contributions as inputs and producing a versioned, traceable, and auditable workflow in which each contribution remains isolated and compatible with the rest of the DEE.

B. Final ECLIPSE Pipeline Architecture

The eight blocks described above assemble into the ECLIPSE pipeline, illustrated in Figure 1. The Authoritative Source of Truth anchors the pipeline by providing the system definitions and requirements from which all other blocks operate. The DEE Thread Pipeline groups the core scenario generation activities, with Module Selection and Urban Planning defining the mission configuration and spatial layout, and the Behavior Engine simulating the resulting system interactions. The Verification Loop connects the simulation outputs back to the Authoritative Source of Truth, closing the feedback cycle. In parallel, CAD Models Submission integrates the geometric representations of each active system, and Visualization assembles all outputs into a 3D mission scenario. Collaboration spans the entire pipeline, providing the version control and governance infrastructure that allows all contributors to work concurrently without interference. Together, these blocks form a traceable digital thread from mission requirements to scenario evaluation.

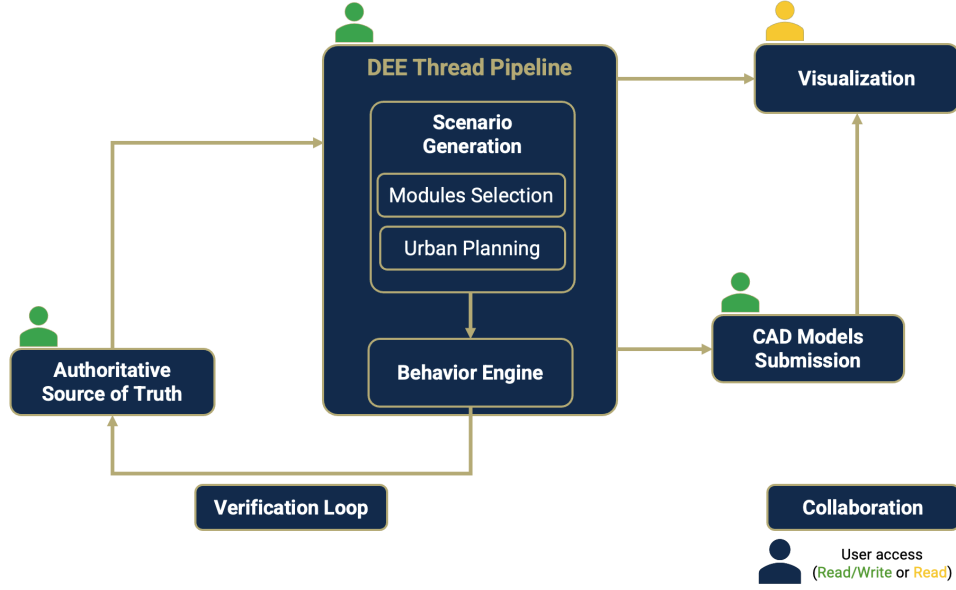


Fig. 1 ECLIPSE Digital Engineering Ecosystem pipeline architecture

IV. Implementation

With the architecture of ECLIPSE defined and its building blocks justified, this section describes how each block was concretely implemented. For each block, the technical choices, data structures, and tools used are detailed, showing how the high-level design decisions translate into a functional pipeline. The implementation follows the same order as the architecture, from the Authoritative Source of Truth through to Visualization and Collaboration, allowing the reader to trace each design decision through to its technical realization.

A. Authoritative Source of Truth: MBSE with SysML v2

The SysML v2 model is developed and maintained in MSOSA, which serves as the primary software environment for model authoring and interoperability across the ecosystem. The model defines subsystem interactions through typed ports and interfaces, including resource flows such as power, regolith, and liquid oxygen, and supports computed attributes and analytical properties directly within the model structure. Requirements are formalized using satisfy relationships that connect each requirement to the subsystem responsible for fulfilling it, establishing end-to-end traceability between stakeholder needs and system design decisions. Simulation outputs are synchronized back into the MSOSA repository through the Requirements Verification Loop, maintaining continuity across the digital thread and supporting continuous verification and validation throughout the pipeline.

B. Module Selection

Following the SysML v2 authoritative model, the module selection step transforms the architecture into a structured JSON and graph representation for downstream analysis, simulation, and visualization. Each system element is parsed as a part containing attributes, hierarchy, ports, metadata, and dimensions. SysML interfaces are converted into directed typed connections that preserve the source port, target port, and exchanged flow, such as power, liquid oxygen, or regolith. The resulting system graph is defined as

$$G = (\mathcal{V}, \mathcal{E}), \quad (1)$$

where nodes represent modules and edges represent interfaces. Before the graph is used by downstream tools, the connections are validated against the port definitions, thus checking that the referenced source and target parts exist, that the required ports exist on those parts, and that the connection direction is consistent with the port definitions.

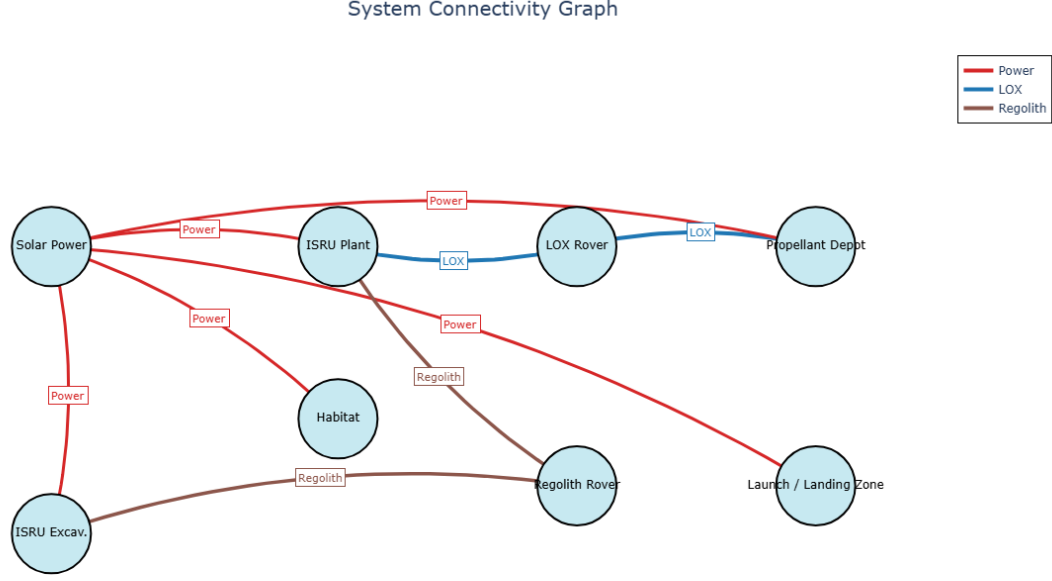


Fig. 2 System connectivity graph generated from the parsed SysML v2 model.

As shown in Fig. 2, this graph enables users to activate mission-specific subsets of the architecture. For an active node set $\mathcal{V}_A \subseteq \mathcal{V}$, the selected subsystem is

$$G_A = (\mathcal{V}_A, \mathcal{E}_A), \quad \mathcal{E}_A = \{e_{ij} \in \mathcal{E} \mid v_i, v_j \in \mathcal{V}_A\}. \quad (2)$$

Thus, only connections between active modules are retained, while hierarchy, ports, flow types, and metadata remain consistent for simulation, visualization, and trade-space analysis.

C. Urban Planning: GIS-Based Site Selection

The GIS workflow is implemented in ArcGIS Pro using LOLA DEM terrain products and illumination datasets. Slope and elevation rasters are derived directly from the DEM at two resolutions: a 2 km average for regional site comparison and a 50 m precision layer for local module placement. Illumination duration rasters and PSR boundary datasets are incorporated as additional planning layers. These environmental layers are combined into a weighted suitability framework in which each criterion is normalized and multiplied by its assigned weight before being summed into an overall suitability score between 0 and 1 for each candidate region. The workflow generates terrain suitability heatmaps for each criterion independently, as well as a composite weighted heatmap, allowing users to inspect the contribution of individual factors to the overall site score.

Once a site is selected, its coordinates become the spatial origin for the module placement phase. Each active module is assigned a placement zone based on its operational role: landing zones are constrained to slopes below 5 degrees, solar infrastructure targets high-illumination areas, and ISRU systems are placed near PSR boundaries for resource access. The user positions each module within the interface, with slope and illumination heatmaps displayed as visual guidance. Module footprints are checked against local terrain constraints to flag any placement in terrain classified as caution or avoid.

The routing algorithm generates traversable paths between modules by solving a weighted shortest path problem on a rasterized terrain graph, where edge weights combine Euclidean distance and local slope angle. The algorithm minimizes total network cost while enforcing a maximum slope threshold of 15 degrees on all traversable edges, producing rover mobility corridors that follow low-slope green and yellow terrain corridors. The user can interactively adjust waypoints within the interface if alternative routing is preferred. The block outputs an updated JSON file enriched with coordinates, orientations, terrain suitability scores, and routing paths for each active module.

D. SE Behavior Engine: Discrete Event Simulation

To model the time-dependent interaction between various active systems, a discrete event simulation was built using the SimPy framework in Python. The active systems are represented as their own class objects with the appropriate attributes, and functions that simulate certain action. For example, an ISRU plant object would contain a processing rate attribute (among many others), and contain a function in order to simulate the extraction of oxygen from lunar regolith that accurately models the time, power, and regolith required to produce a certain amount of oxygen. These different class objects and their attributes are setup and instantiated using data imported from the SysML model (in the form of a JSON file) and linked together using SimPy processes according to a specific case study. The DES is setup to be flexible enough that various systems can be activated or deactivated in order to simulate different scenarios with different systems. Additionally, a few parameters were also made easily accessible to the end user so that they could test the impact of varying their values (such as number of rovers and ISRU plants, processing rates, rover speed, the distance from the excavation site to the ISRU plant, etc). Once the user runs the DES given data from the SysML model and the appropriate parameter values, the DES outputs two JSON files. The first JSON file outputted is a summary of the results and will be referred to as the results JSON file. This file contains a high level overview of the status of each system at the end of the simulation. This is useful for a quick check on the overall results of the simulation. The second file outputted by the DES is a log of all of the attributes of every system at every time step in the DES and will be referred to as the log JSON file. This file is useful since it helps enable the visualization of the simulation, allows one to closely inspect the actions of each system across every point in time, and also enables the requirements for each system to be checked at every time step to make sure they were never violated.

E. Requirements Verification Loop

The verification loop is implemented as a five-step automated process. First, the Behavior Engine produces time-history outputs at each simulation timestep, recording key performance metrics and state data for every active system. Second, these outputs are exported into a log JSON file structured to preserve the system identifier, attribute name, value, and timestamp for each recorded entry. Third, the log JSON is imported into the SysML v2 model in MSOSA, where each JSON attribute is mapped to its corresponding model element using the system identifier as the matching key. Fourth, at each timestep, the mapped attribute value is evaluated against the requirement constraint associated with that model element through the satisfy relationship, and a pass or fail result is recorded. Fifth, the verification output is compiled into a structured report that identifies which requirements are satisfied, which are violated, at which timesteps violations occur, and which systems are involved.

This closed-loop process allows any architectural change made in the SysML model to be propagated through the pipeline, re-simulated, and re-verified automatically, without manual intervention at any stage of the verification chain.

F. CAD Submission

After module selection, described in Section IV.B, the activated JSON assets are passed to the CAD submission step. This stage links each selected module with its corresponding CAD model, currently supporting .usd, .usda, .usdc, and .usdz files. For each active module, the pipeline reads the part-level JSON file and retrieves its identifier, dimensions, position, rotation, attributes, material reference, geometry reference, and ports.

The output is a unified submission manifest that connects the semantic model information with the available geometry representation. The manifest also reports which modules have valid CAD files and which remain missing. If a CAD model is not available in the database, the user can upload the corresponding geometry file and link it to the selected module. Also, at this step the user can inspect all available information for each module before downstream visualization. Future work will extend this step to additional CAD formats.

G. Visualization: USD Scene Builder and Omniverse

The visualization stage prepares the selected architecture for inspection in Omniverse by converting the submission manifest, connection data, and simulation outputs into USD scene layers. Using Python, a routine called *USD Scene Builder* consumes the final enriched JSON and generates the main scene layer, `scene.usd`, together with a separate waypoint layer, `waypoints.usd`, derived from the urban planning analysis and the optimal trajectories of each module. The scene layer instantiates each active module under a common USD stage, applies its position and rotation, and references the available CAD geometry. It also preserves module metadata, dimensions, attributes, and material

information as USD attributes. Connection paths and waypoints are written as USD files, allowing flows such as power, liquid oxygen, and regolith to be visualized alongside the physical layout.

The resulting USD files are then loaded into NVIDIA Omniverse, which acts as the visualization and execution environment of the DEE. Through the Omniverse extension, users can pull the GitHub repository, load `scene.usd` and `waypoints.usd`, and play the mission scenario using the log output from the SE Behavior Engine. This allows stakeholders to review module placement, connection paths, resource movement, and scenario results in a shared 3D environment. Modifying a DES lever triggers a new simulation run and reloads the updated scene in Omniverse, producing a revised visualization without code modification. The telemetry dashboard, which will display key mission variables during playback, is planned for completion.

H. Collaboration

The ECLIPSE pipeline is hosted in a shared GitHub repository that stores all artifacts produced and consumed at each pipeline step, including SysML files, JSON files, USD files, DES scripts, and CAD models. Each pipeline step corresponds to a defined folder structure within the repository, allowing contributors to locate and update their respective artifacts without interacting with unrelated parts of the codebase.

GitHub Actions are configured to trigger automated pipeline runs at each commit to the main branch, ensuring that updates to any artifact are automatically propagated through the downstream steps without manual intervention. Pull requests serve as the contribution and conflict-resolution mechanism: each contributor works on an isolated branch, and changes are merged into the main branch only after review, preventing unvalidated contributions from disrupting the shared pipeline state.

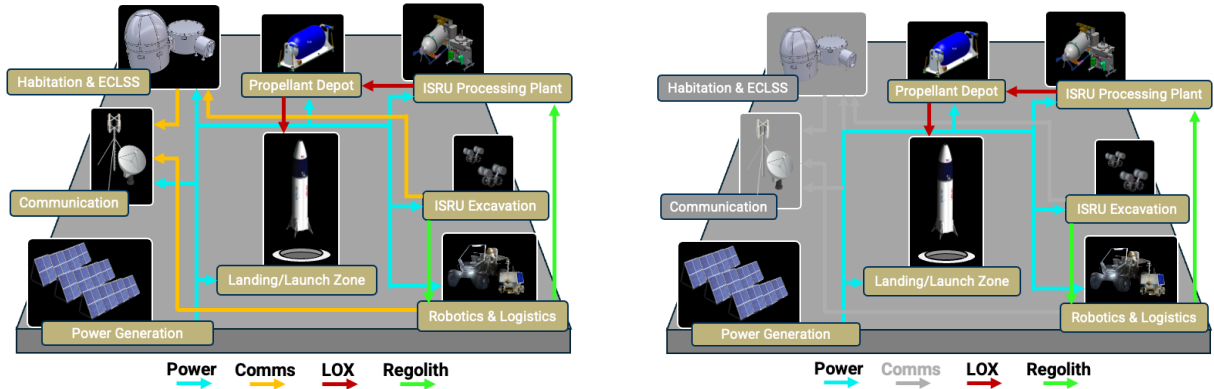
The HTML/JS web portal provides a single end-to-end interface for the end user, abstracting the underlying GitHub repository and tool interactions behind a unified graphical environment. Each pipeline step is accessible through a dedicated tab within the portal, allowing mission planners, GIS architects, hardware suppliers, systems engineers, and mission evaluators to contribute to their respective steps without requiring direct access to the underlying tools or file structures. The web portal interface is presented in detail in the Results section.

V. Results

This section presents the results obtained by applying the ECLIPSE pipeline to the Lunar Spaceport-1 case study. The demonstration follows the pipeline from end to end, from system configuration through site selection, simulation, and visualization, and is used to evaluate the extent to which the pipeline delivers on its three research questions.

A. Case Study: Lunar Spaceport-1

The Lunar Spaceport-1 concept encompasses eight systems: a power generation unit, a communications array, a habitation and ECLSS module, an ISRU processing plant, a robotics and logistics unit, a landing and launch zone, a propellant depot, and an ISRU excavation site. These systems exchange power, communications signals, liquid oxygen, and regolith across the spaceport. As illustrated in Figure 3(a), the full Concept of Operations defines all potential interactions between systems. For this proof-of-concept demonstration, the scope is limited to the ISRU-specific interactions: as shown in Figure 3(b), the active flows are power, LOX, and regolith, while communications-related interactions are kept inactive.



(a) Full Lunar Spaceport-1 Concept of Operations

(b) ISRU-focused case study scenario

Fig. 3 Lunar Spaceport-1 Concept of Operations: (a) full system interactions and (b) active ISRU interactions for the case study, with power, LOX, and regolith flows highlighted and communications interactions deactivated.

B. End-to-End Pipeline Demonstration

Figure 4 presents a composite view of the ECLIPSE web interface across the six main pipeline steps, illustrating how the user interacts with the system from configuration to visualization.

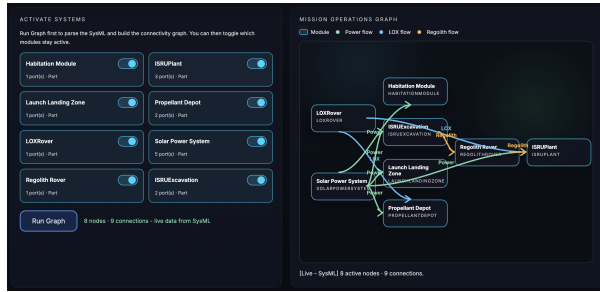
In panel (a), the user accesses the Mission Network Activation interface, which displays a graph representation of all systems defined in the SysML model and their interfaces. The user selects the systems that will participate in the ISRU scenario by activating the relevant nodes, and the graph updates automatically to reflect only the active systems and their remaining interfaces, giving immediate visual feedback on how the configuration changes affect system connectivity. Inactive systems and their interfaces are automatically excluded from all downstream steps. The output is an enriched JSON file containing the active systems, their attributes, and their interfaces.

In panel (b), the user accesses the CAD Submission interface, where the SysML attributes of each active system are reviewed and the corresponding 3D models are uploaded and previsualized. Each CAD model is automatically matched to its corresponding system in the JSON file, and the updated JSON now contains system attributes, geometrical models, positions, and orientations for all active systems.

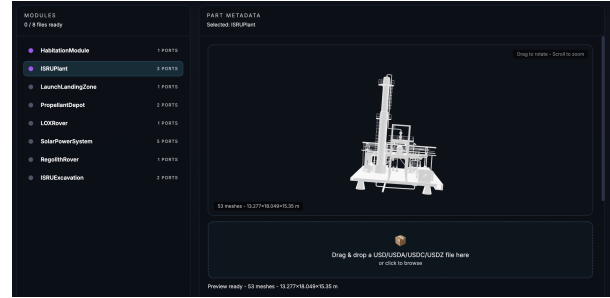
In panel (c), the user accesses the site selection interface, which displays the nine Artemis Candidate Landing Regions at the lunar south pole scored against the weighted terrain criteria. The user can click on each candidate site to inspect its slope, illumination, PSR proximity, and elevation scores, and can compare two sites side by side before selecting the most appropriate location for the mission.

In panel (d), the user accesses the module placement interface, where the active systems are positioned within the selected site using local slope and illumination heatmaps as guidance. The user places each module and confirms its orientation, and the routing algorithm automatically generates the optimal paths between modules by jointly minimizing total network distance and terrain slope. The user can manually adjust waypoints if the generated routes are not satisfactory.

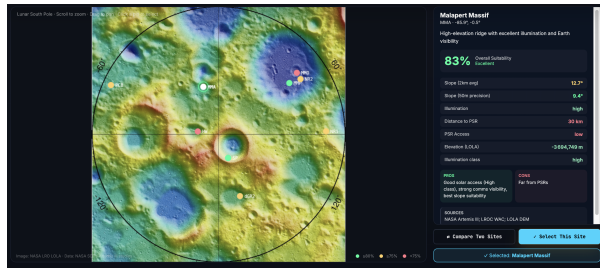
In panel (e), the user accesses the Behavior Engine interface, where the DES parameters are configured through sliders. The user sets the simulation levers including excavation distance, number of rovers and ISRU plants, processing rate, and LOX transfer threshold. Resource connections between systems are visualized directly in the interface, and the most important simulation outputs are displayed for immediate mission evaluation. At this stage, all required inputs are gathered and the complete scenario is sent to NVIDIA Omniverse for visualization.



(a) Module Selection



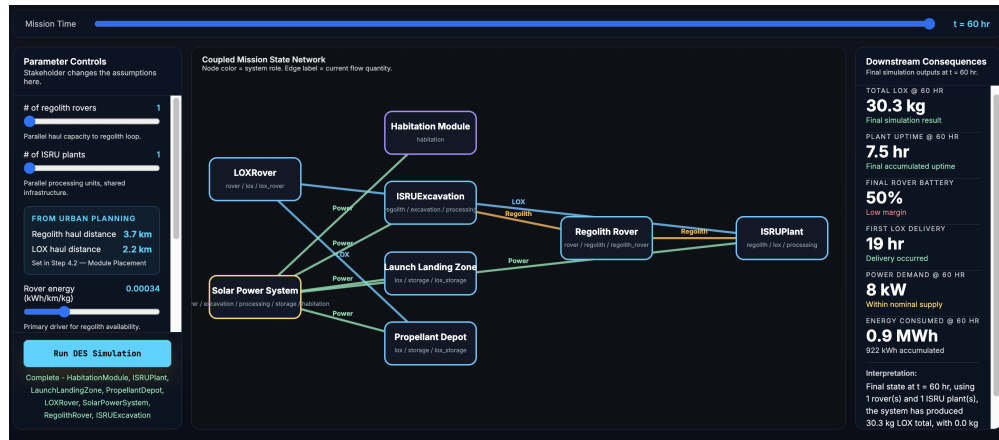
(b) CAD Submission



(c) Site Selection



(d) Module Placement



(e) Behavior Engine Configuration

Fig. 4 ECLIPSE web interface across the six main pipeline steps.

C. GIS Site Selection

The GIS workflow was applied to the nine NASA Artemis Candidate Landing Regions at the lunar south pole. Table 3 summarizes the suitability scores obtained for each candidate site under the weighted criteria defined for the ISRU scenario, which prioritizes terrain stability, power access, and proximity to excavable resources. Malapert Massif achieved the highest overall score of 0.83, driven by its very high illumination conditions, relatively low average slope of 9.41 degrees, and strong communication visibility associated with its elevated terrain. Slater Plain and Mons Mouton Plateau followed closely, with scores of 0.81 and 0.80 respectively, offering favorable terrain stability and infrastructure expansion potential despite lower PSR proximity scores. A different mission type with different operational priorities would produce a different weighting and potentially a different site selection outcome.

Table 3 Candidate Landing Region Suitability Results

Site	Avg. Slope (°)	Illumination	PSR Proximity	Suitability Score
Malapert Massif	9.41	Very High	Low	0.83
Slater Plain	21.81	Low	Very High	0.81
Mons Mouton Plateau	8.60	High	Low	0.80
Nobile Rim 1	16.69	High	Medium	0.79
Peak near Cabeus B	10.95	Medium	High	0.78
Nobile Rim 2	18.41	Medium	Medium	0.77
de Gerlache Rim 2	36.25	Medium	Very High	0.76
Haworth	31.07	Low	Very High	0.72
Mons Mouton	38.05	Medium	Medium-Low	0.68

Following site selection, the module placement phase positioned each active system within Malapert Massif according to local terrain constraints, as illustrated in Figure 5. The Landing Pad was placed on the flattest contiguous green patch, with slopes below 5 degrees, ensuring safe landing conditions and heavy load handling. The Habitation Module was positioned on a green-yellow bench with slopes between 0 and 10 degrees, providing stable construction ground and central access to all other modules. The Propellant Depot was placed on a nearby yellow-green shelf, keeping it close to the landing pad while remaining safely offset from the habitat. The Communications Relay was positioned on an elevated green ridge to maximize line-of-sight coverage across the terrain. The ISRU Excavation site was located on the yellow-orange fringe, closer to rugged terrain for resource access while remaining sufficiently distant from crewed areas for safety and dust management. Rover routes follow the low-slope green and yellow corridors, minimizing traversal of steep terrain above 15 degrees.

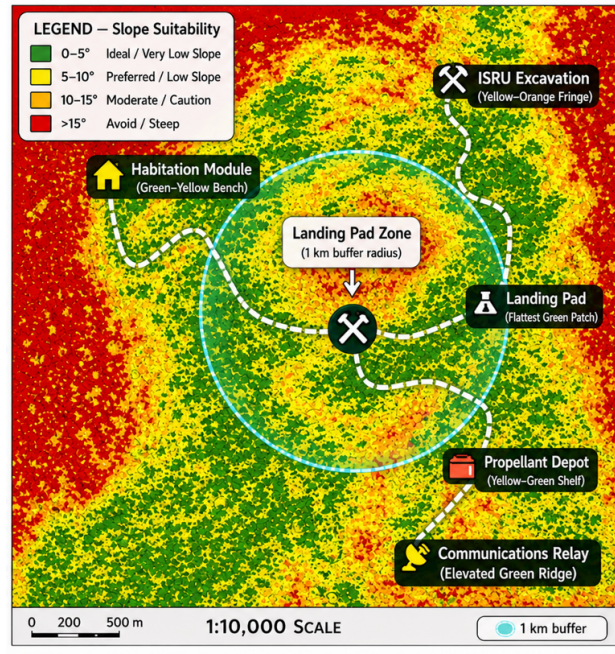


Fig. 5 Module placement and rover routing at Malapert Massif

D. Discrete Event Simulation

The Behavior Engine was run on the ISRU scenario with the following configuration: a solar power generation unit producing 100 kW with a 500 kWh battery, a regolith rover carrying 4,000 kg per trip over a 0.5 km excavation distance derived from the Urban Planning step, an LOX rover, five ISRU processing plants, a habitat module drawing 5 kW, a communications array drawing 2 kW, and a launch and landing zone. Table 4 summarizes the key outputs of the simulation.

Table 4 Discrete Event Simulation Results

System	Parameter	Value
5*ISRU Plant	LOX Stored	10.11 kg
	Energy Consumed	3358.82 kWh
	Total Operational Hours	27.50 hr
	Regolith Received	44,000.00 kg
	Total LOX Production	111.23 kg
3*Solar Power System	Total Generated	5900.00 kWh
	From Battery	0.00 kWh
	Battery Charge	500.00/500.00 kWh
Habitat-1	Energy Consumed	315.00 kWh
CommArray-1	Energy Consumed	128.00 kWh
2*LZ-Alpha	LOX Stored	101.12 kg
	Energy Consumed	321.04 kWh
4*Regolith Cargo Rover	Total Distance	12.00 km
	Energy Consumed	16.32 kWh
	Battery Charge	83.68/100.00 kWh
	Current Load	4,000.00 kg
4*LOX Cargo Rover	Total Distance	1.00 km
	Energy Consumed	0.03 kWh
	Battery Charge	99.97/100.00 kWh
	Current Load	0.00 kg

Several observations can be drawn from these results. The solar power system generated a total of 5,900 kWh over the simulation duration, maintaining the battery at full capacity throughout without drawing from storage at any point. This indicates that the 100 kW generation capacity is well-sized for the combined power demand of the active systems in this ISRU configuration, with the ISRU plants consuming the large majority of available energy at 3,358.82 kWh, followed by the launch and landing zone at 321.04 kWh, the habitat at 315.00 kWh, and the communications array at 128.00 kWh. The rover batteries, both for regolith and LOX transport, remained close to full charge throughout the mission, confirming that the excavation distance of 0.5 km derived from the Urban Planning step does not impose a significant energy burden on the logistics chain.

The ISRU processing chain processed a total of 44,000 kg of regolith over 27.5 operational hours, producing 111.23 kg of liquid oxygen in total. Of this, 101.12 kg was successfully transferred to the launch and landing zone, representing a transfer efficiency of approximately 91%. The remaining LOX was retained in intermediate storage within the ISRU plant at the end of the simulation window.

These results also illustrate the value of connecting Urban Planning outputs directly to the DES inputs. The excavation distance and routing paths derived from the module placement step were automatically propagated into the simulation parameters, ensuring that rover travel times and energy consumption reflected the actual terrain layout rather than abstract assumptions. This terrain-aware configuration is a direct demonstration of the end-to-end digital thread connecting the GIS, simulation, and verification steps within the ECLIPSE pipeline.

E. Visualization

The complete Lunar Spaceport-1 ISRU scenario was loaded and played back in NVIDIA Omniverse. The USD Scene Builder consumed the final enriched JSON, incorporating the system attributes from the SysML model, the spatial layout from Urban Planning, the simulation log from the Behavior Engine, and the CAD models submitted by hardware stakeholders, and generated the scene.usd and waypoints.usd files. The Omniverse Extension provides the end user with the ability to pull the GitHub repository, load the scene, and play back the mission scenario according to the DES log.

As shown in Figure 6, the resulting visualization places each active system at its terrain-aware location within Malapert Massif, with rover routing paths connecting the modules. Stakeholders can inspect the infrastructure layout in the actual lunar terrain, observe the movement of rovers between systems, and follow the time-evolving resource flows between the ISRU plant, the propellant depot, and the launch and landing zone. This gives all stakeholders, regardless of their technical background, a direct spatial and temporal understanding of the mission that the numerical outputs of Table 4 alone cannot convey.



Fig. 6 ECLIPSE mission scenario played back in NVIDIA Omniverse, showing the Lunar Spaceport-1 ISRU infrastructure placed within the lunar terrain with rover routing paths and active system interactions.

VI. Discussion

The results presented in the previous section demonstrate that ECLIPSE successfully instantiates a requirements-driven, collaborative, and visualization-based pipeline for lunar base architecting. This section interprets these results in light of the three research questions that motivated the work, and identifies the current limitations of the approach.

A. Answers to the Research Questions

RQ.1: How can a lunar scenario generation pipeline be designed to be entirely requirements-driven and modular by design, so that any system configuration can be generated, simulated, and evaluated through a single generalized optimization process that adapts to the mission at hand?

ECLIPSE addresses this question through modularity at two levels. At the requirements level, the Authoritative Source of Truth defines the full set of systems available for any mission scenario: only systems formally defined in the SysML model can subsequently be considered, establishing the boundaries of the design space. At the execution level, Module Selection allows the user to scale down from this full set by selecting which systems actually participate in a given scenario, without modifying any code. This configuration is then propagated automatically through the rest of the pipeline via the JSON data backbone: Urban Planning positions only the selected modules, hardware stakeholders can

only submit CAD models for the activated systems, and the Behavior Engine simulates only the interactions between them. The case study on Lunar Spaceport-1 demonstrated this capability on an ISRU-focused scenario.

RQ.2: How can independent stakeholders contribute their systems and data into a shared pipeline while guaranteeing traceability, metadata integrity, and non-interference between contributions?

ECLIPSE addresses this question through its Collaboration block. GitHub provides isolated development branches and pull requests as a conflict-resolution mechanism, ensuring that no contributor can overwrite another's work without review. This is illustrated concretely by the CAD Submission block, where hardware stakeholders submit their 3D models independently through the web interface, without requiring access to the rest of the pipeline or exposure of other contributors' work. The complete commit history constitutes a full audit trail of every design decision made throughout the pipeline. Shared open standards, namely SysML for architecture, JSON for engineering data, and OpenUSD for geometry and scene, ensure that metadata flows through the pipeline without modification or loss across organizational boundaries, without requiring stakeholders to use the same proprietary tools. Intellectual property is protected because each organization contributes only through the standards interface, without exposing internal tooling or proprietary data formats.

RQ.3: How can we go beyond quantitative MoEs to provide a spatial and temporal evaluation of a lunar mission that is directly interpretable by all stakeholders?

ECLIPSE addresses this question through the combination of GIS-based Urban Planning, the Behavior Engine, and the Visualization block, which together produce an evaluation environment that quantitative metrics alone cannot replicate. The DES outputs of Table 4 provide a precise account of system performance, but they do not reveal where the ISRU excavation site is relative to the propellant depot, how far the regolith rovers travel between trips, or how the power budget evolves as multiple systems draw concurrently. These spatial and temporal dimensions of the mission are only accessible through the Omniverse visualization, where each system is placed at its terrain-aware location within Malapert Massif and the mission scenario unfolds according to the DES log.

This matters because mission evaluation is not only a numerical exercise. A stakeholder reviewing Table 4 can confirm that 111.23 kg of LOX were produced, but cannot immediately assess whether the infrastructure layout is operationally coherent, whether rover routes avoid steep terrain, or whether the propellant depot is positioned for efficient LOX transfer. The Omniverse visualization makes these questions directly answerable by giving all stakeholders, regardless of their technical background, a shared view of the mission that connects performance data to physical reality. This represents a qualitative shift in how mission architects, hardware suppliers, and mission evaluators can collectively assess and compare alternative configurations.

B. Limitations

Several limitations of the current implementation should be acknowledged. While the pipeline structure is generalizable, the Behavior Engine models are currently written for the ISRU mission type and would require adaptation to support significantly different operational scenarios. The Behavior Engine also operates at intermediate fidelity: it captures the event-driven logic of surface operations but does not model thermal effects, dust dynamics, or system failure modes, all of which would affect real mission performance. On the collaboration side, the current implementation relies on a GitHub-based workflow that requires contributors to interact asynchronously; real-time multi-user access within the web portal has not yet been implemented. Finally, the current GIS workflow primarily relies on LOLA DEM data and focuses on terrain suitability at a regional planning scale rather than detailed operational-scale surface engineering. The routing workflow currently prioritizes slope and distance but does not yet account for additional factors such as dynamic hazards or temporal environmental changes. Additionally, the current implementation focuses on an ISRU-centered mission scenario, and broader mission architectures would require further expansion of the simulation and planning logic.

VII. Conclusions and Future Work

A. Summary

This paper presented ECLIPSE, a Digital Engineering Ecosystem developed to address the collaborative, modular, and visualization-based challenges of lunar base architecting. By assembling eight functional blocks into a single requirements-driven pipeline, ECLIPSE demonstrates that it is possible to go from mission requirements to a spatially grounded, simulated, and visually evaluated mission scenario within a shared digital environment accessible to multiple independent stakeholders.

Three fundamental contributions emerge from this work. First, the modularity of the pipeline at both the requirements and execution levels allows any ISRU mission configuration to be generated and evaluated through a single generalized process, reducing what would otherwise require months of dedicated effort to a streamlined interactive workflow. Second, the use of open standards and version control infrastructure enables independent organizations to contribute concurrently without interference, with full traceability and intellectual property safeguards, addressing a collaboration gap that has long limited the architecting of complex lunar missions. Third, the integration of NVIDIA Omniverse as the evaluation environment moves mission assessment beyond quantitative Measures of Effectiveness, giving stakeholders a direct spatial and temporal understanding of how the mission unfolds that numbers alone cannot provide.

Taken together, these contributions represent a first demonstration that a unified, requirements-driven DEE pipeline for collaborative lunar base architecting is both feasible and practically implementable within the LUNaverse Phase 0 framework.

B. Future Work

Building on the foundation established by ECLIPSE, four directions are identified for future development. First, the Behavior Engine should be extended with higher-fidelity physics-based models covering power systems, thermal effects, and surface mobility, in order to increase the realism of simulated mission scenarios and broaden the range of mission types that can be evaluated. Second, the system scope should be expanded beyond ISRU to include crewed operations, logistics chains, failure modes, and additional mission elements, demonstrating the generalizability of the pipeline across a wider range of lunar surface activities. Third, the collaboration infrastructure should evolve toward a scalable multi-user web portal with real-time concurrent access, scenario saving, and comparison tools, removing the current dependency on asynchronous GitHub-based workflows. Fourth, the Visualization block should be developed further toward time-evolving, physics-accurate mission rendering and immersive VR exploration within Omniverse, completing the vision of a fully interactive mission evaluation environment.

Acknowledgments

The authors would like to thank the Georgia Tech Aerospace Systems Design Laboratory and the LUNaverse program for their support throughout this work.

References

- [1] “National Space Policy of the United States of America,” Tech. rep., The White House, December 2020.
- [2] National Science and Technology Council, “National Cislunar Science and Technology Action Plan,” Tech. rep., Executive Office of the President of the United States, December 2024.
- [3] Kratsios, M. J., “Memorandum for the Heads of Executive Departments and Agencies,” Tech. rep., Office of Science and Technology Policy, 2025.
- [4] Mavris, D. N., Robertson, B., Balchanos, M., and Whorton, M., “The Georgia Tech Cislunar Architecture Study,” *AIAA ASCEND 2022 Conference*, American Institute of Aeronautics and Astronautics, Las Vegas, Nevada, USA, 2022. <https://doi.org/10.2514/6.2022-4332>.
- [5] NASA, “NASA’s Lunar Exploration Program Overview,” Tech. rep., National Aeronautics and Space Administration, 2020.
- [6] Elrick, T., Hawkins, J., Balchanos, M., and Mavris, D. N., “Cislunar Environment Architecting: A Tabletop Approach,” *AIAA ASCEND 2022 Conference*, American Institute of Aeronautics and Astronautics, Las Vegas, Nevada, USA, 2022. <https://doi.org/10.2514/6.2022-4363>.

- [7] Boyer, P., et al., “Integrated Parametric Framework for Designing In-Situ Resource Utilization Capabilities in a Lunar Base Architecture,” *AIAA SciTech Forum*, American Institute of Aeronautics and Astronautics, 2026.
- [8] Ramkumar, A., et al., “An Integrated MDAO Enabled MBSE Approach for Sizing and Optimizing Lunar Power Systems,” *AIAA SciTech Forum*, American Institute of Aeronautics and Astronautics, 2026.
- [9] Mehta, A., et al., “Parametric Modeling of a Lunar Base Architecture with a Focus on Power Systems Approaches,” *AIAA ASCEND 2024 Conference*, American Institute of Aeronautics and Astronautics, 2024.
- [10] Georgia Tech ASDL, “Moon Operations for Resupply Payload Handling (MORPH),” Tech. rep., Georgia Institute of Technology, 2024. Grand Challenge 2024.
- [11] Georgia Tech ASDL, “Design Environment for Lunar Operations and Systems Architecting (DELOS),” Tech. rep., Georgia Institute of Technology, 2024. Grand Challenge 2024.
- [12] Georgia Tech ASDL, “Permanent Infrastructure and Operations for Next-generation Earth-to-Moon Establishment and Research (PIONEER),” Tech. rep., Georgia Institute of Technology, 2025. Grand Challenge 2025.
- [13] U.S. Department of Defense, “Digital Engineering Strategy,” Tech. rep., Office of the Deputy Assistant Secretary of Defense for Systems Engineering, June 2018.
- [14] Friedenthal, S., Moore, A., and Steiner, R., *A Practical Guide to SysML: The Systems Modeling Language*, 3rd ed., Morgan Kaufmann, 2014.
- [15] Grieves, M., “Digital Twin: Manufacturing Excellence Through Virtual Factory Replication,” *White Paper*, 2014.
- [16] Open Solar System Foundation, “Open Solar System Planetary Spatial Data Infrastructure (OSS-PSDI) Certification Framework,” Tech. rep., OSSF, 2024. OSSF Whitepaper.